

Programação Orientada a Objetos - Lista 2

1. Objetivo

Desenvolver um sistema simples em **Java** para gerenciar um zoológico, com suporte a diferentes tipos de animais.

O sistema deverá utilizar os conceitos de **herança** e **polimorfismo**, permitindo que objetos de diferentes subclasses sejam armazenados e manipulados por meio do tipo `Animal`.

O sistema deve permitir **adicionar, listar e remover animais**, além de emitir seus sons e testar suas habilidades.

2. Estrutura do Sistema

2.1 Classe: `Animal`

Crie uma classe chamada `Animal`, que servirá como superclasse para os diferentes tipos de animais do sistema.

A classe deve conter os seguintes atributos:

```
long id
String nome
int idade
double peso
ArrayList<String> habilidades
```

Cada animal deve possuir um **ID único**, informado no momento do cadastro. O sistema não deve permitir que dois animais possuam o mesmo ID.

A classe deve possuir um construtor que receba:

```
Animal(long id, String nome, int idade, double peso)
```

O atributo `habilidades` deve ser inicializado como uma lista vazia.

A classe também deve possuir o seguinte método:

```
String emitirSom()
```

Na classe `Animal`, esse método deve retornar:

```
som desconhecido
```

As subclasses deverão **sobrescrever** esse método para retornar o som correspondente ao tipo de animal.

Também implemente:

```
String realizarHabilidade(String habilidade)
```

O método deve verificar se o animal possui a habilidade informada em sua lista de habilidades.

Caso possua, deve retornar:

<nome> consegue <habilidade>

Caso não possua, deve retornar:

<nome> não consegue <habilidade>

Implemente também os métodos get necessários para acessar os atributos dos animais.

2.2 Subclasses: Gato, Cachorro e Passaro

Crie as classes:

```
Gato
Cachorro
Passaro
```

Todas elas devem **herdar da classe Animal**.

Cada subclasse deve possuir um construtor com os seguintes parâmetros:

```
Gato(long id, String nome, int idade, double peso)
```

```
Cachorro(long id, String nome, int idade, double peso)
```

```
Passaro(long id, String nome, int idade, double peso)
```

Os construtores das subclasses devem utilizar o construtor da superclasse para inicializar id, nome, idade e peso.

Cada tipo de animal deverá possuir automaticamente uma habilidade específica:

```
Gato      -> agilidade
Cachorro  -> farejar
Passaro   -> voar
```

Assim, ao criar um objeto dessas classes, sua habilidade correspondente deverá ser adicionada à lista habilidades.

Cada subclasse também deverá **sobrescrever** o método:

```
String emitirSom()
```

Os retornos devem ser:

```
Gato      -> miau
Cachorro  -> au
Passaro   -> piu
```

Exemplo:

```
Animal animal = new Cachorro(1, "Rex", 5, 20.5);

System.out.println(animal.emitirSom());
```

Saída:

```
au
```

Observe que a variável é do tipo `Animal`, mas o objeto criado é um `Cachorro`. Ao executar `emitirSom()`, deve ser executado o método definido na classe `Cachorro`.

2.3 Classe: Zoologico

A classe Zoologico será responsável por gerenciar todos os animais cadastrados.

Ela deve possuir o seguinte atributo:

```
ArrayList<Animal> animais
```

A mesma lista deverá armazenar objetos das classes Gato, Cachorro e Passaro.

Por exemplo:

```
animais.add(new Gato(...));  
animais.add(new Cachorro(...));  
animais.add(new Passaro(...));
```

Implemente os seguintes métodos:

```
boolean adicionarAnimal(Animal animal)
```

Adiciona um animal à lista. O método deve verificar se já existe um animal com o mesmo ID. Caso o ID ainda não exista, o animal deve ser adicionado e o método deve retornar true. Caso o ID já esteja cadastrado, o animal não deve ser adicionado e o método deve retornar false.

```
ArrayList<Animal> listarAnimais()
```

Retorna a lista contendo todos os animais cadastrados.

```
Animal buscarAnimal(long id)
```

Procura um animal pelo ID. Caso encontre, retorna o objeto correspondente. Caso não encontre, retorna null.

```
boolean removerAnimal(long id)
```

Remove o animal que possui o ID especificado. Retorna true caso o animal seja encontrado e removido, e false caso não exista um animal com o ID informado.

3. Menu Interativo do Sistema

Implemente um menu interativo utilizando Scanner no método main. O menu deve permitir que o usuário interaja com o zoológico por meio do terminal.

As opções obrigatórias são:

1. Adicionar animal
2. Listar todos os animais
3. Remover animal
4. Emitir som de um animal
5. Testar habilidade de um animal
6. Sair

Opção 1 - Adicionar animal

O usuário deverá escolher o tipo de animal:

1. Gato
2. Cachorro
3. Passaro

Depois, deverá informar:

ID
Nome
Idade
Peso

O programa deverá criar o objeto correspondente e adicioná-lo ao Zoologico.

Exemplo:

```
Animal animal = new Cachorro(id, nome, idade, peso);  
zoologico.adicionarAnimal(animal);
```

Opção 2 - Listar todos os animais

Percorra o `ArrayList<Animal>` e mostre as informações de todos os animais cadastrados. A mesma lista deve conter objetos de diferentes subclasses.

Exemplo:

```
for (Animal animal : zoologico.listarAnimais()) {  
    System.out.println(animal.getNome());  
}
```

Opção 3 - Remover animal

Solicite o ID do animal e utilize:

```
zoologico.removerAnimal(id);
```

Informe ao usuário se a remoção foi realizada com sucesso.

Opção 4 - Emitir som de um animal

Solicite o ID do animal e utilize `buscarAnimal()`. O método `emitirSom()` deverá ser chamado utilizando uma referência do tipo `Animal`:

```
Animal animal = zoologico.buscarAnimal(id);  
  
if (animal != null) {  
    System.out.println(  
        animal.getNome() + " diz " + animal.emitirSom()  
    );  
}
```

O método executado dependerá do **tipo real do objeto armazenado**.

Por exemplo, supondo que existam três objetos armazenados como `Animal`:

```
Rex      -> objeto Cachorro  
Mingau   -> objeto Gato  
Piu-Piu  -> objeto Passaro
```

A chamada de `emitirSom()` deverá produzir resultados diferentes:

```
Rex diz au  
Mingau diz miau  
Piu-Piu diz piu
```

Opção 5 - Testar habilidade de um animal

Solicite:

ID do animal
Habilidade

Depois, utilize:

```
Animal animal = zoologico.buscarAnimal(id);

if (animal != null) {
    System.out.println(
        animal.realizarHabilidade(habilidade)
    );
}
```

Exemplos:

Rex consegue farejar
Rex não consegue agilidade

Mingau consegue agilidade
Mingau não consegue voar

Piu-Piu consegue voar

4. Requisitos de Implementação

1. Implemente a classe `Animal` com todos os atributos, construtor e métodos especificados.
2. Implemente `Gato`, `Cachorro` e `Passaro` utilizando **herança** a partir da classe `Animal`.
3. Utilize `super(...)` nos construtores das subclasses para inicializar os atributos herdados.
4. Sobrescreva o método `emitirSom()` em `Gato`, `Cachorro` e `Passaro`.
5. Implemente a classe `Zoologico` utilizando `ArrayList<Animal>` para armazenar todos os tipos de animais.
6. Não crie uma lista separada para cada tipo de animal. Todos os objetos devem ser armazenados na mesma coleção de `Animal`.
7. Garanta que não existam dois animais com o mesmo ID.
8. Utilize referências do tipo `Animal` para buscar e manipular os objetos cadastrados.
9. Ao chamar `emitirSom()` através de uma referência `Animal`, o comportamento deve depender da subclasse do objeto armazenado.
10. Implemente o menu utilizando `Scanner`.

5. Exemplo de Execução

Considere os seguintes animais cadastrados:

ID: 1
Tipo: Cachorro
Nome: Rex
Idade: 5
Peso: 20.5

ID: 2
Tipo: Gato
Nome: Mingau
Idade: 3
Peso: 4.2

ID: 3
Tipo: Passaro
Nome: Piu-Piu
Idade: 2
Peso: 0.4

Algumas possíveis saídas do sistema são:

Rex diz au
Mingau diz miau
Piu-Piu diz piu

Rex consegue farejar
Rex não consegue agilidade

Mingau consegue agilidade
Piu-Piu consegue voar

Animal removido: true